

Lab 5 ? Model Registry Governance

Track A (tabular fraud-detection) · Week 5 · DS5619 Machine Learning Systems Operations

Setup

```
`bash
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/candidate_a/{model.json,metrics.json}` and `data/candidate_b/{model.json,metrics.json}` with values generated deterministically from your student ID ? same structure as everyone else's (candidate_a always fails the 0.70 f1 production bar, candidate_b always clears it ? otherwise the governance-gate point wouldn't demonstrate anything), but different actual threshold/metric numbers. Two students never get the same data.

Record your `--student-id` value in `NOTES.md`. The grader re-runs `generate_for_student.py` with the ID you recorded and diffs the result against what you committed, and checks that your `registry_summary.json`'s production metrics actually match YOUR candidate_b, not a generic or shared example.

Learning objective

This week's lecture covered the model registry as an artifact store, model cards as the one-page governance record, and why the registry ? not a Slack thread or someone's memory ? is the answer to "what's actually in production." You'll build a minimal local model registry that enforces those two governance rules as code, not just policy: no promotion to Production without a complete model card, and no promotion without metrics that clear a quality bar.

This week's lecture also named where the problem starts: a hyperparameter search or AutoML run producing many near-identical candidates (like candidate_a/candidate_b here, just two of them) ? explicitly scoped as model-development work this course doesn't teach as a lab, because the registry is what happens after that search ends, which is exactly what you're building.

Files

- `src/mini_model_registry.py` ? implement the four # TODO functions.
- `src/run_pipeline.py` ? complete driver script. Don't edit.
- `model_card_fields.json` ? fill in with real content before running the pipeline (it will refuse to run while any TODO remains).
- `data/candidate_a/`, `data/candidate_b/` ? your two personalized model candidates + their metrics, generated above (don't hand-edit).

Background

`data/candidate_a/` and `data/candidate_b/` are two already-trained candidate models for a fraud-detector (deliberately simple: a single amount threshold), each with its own `metrics.json`. One clears production quality bar, one doesn't ? you won't be told which until you run the pipeline and see the registry enforce it.

Your task

Part 1 ? src/mini_model_registry.py (four functions marked # TODO, each has a full docstring spec)

- register_model(name, model_path, metrics, registry_dir) ? the artifact store: version a model file + its metrics, initial stage "None".
- generate_model_card(name, version_id, card_fields, registry_dir) ? the governance record: reject anything with a missing or TODO-containing field, otherwise write it.
- promote_model(name, version_id, target_stage, registry_dir) ? the gate: Production requires a card AND metrics["f1"] >= PRODUCTION_F1_THRESHOLD; a successful promotion to Production archives whichever version was previously there.
- get_production_model(name, registry_dir) ? what's actually in production, right now, no memory required.

Part 2 ? model_card_fields.json (fill in real content)

Replace every "TODO: ..." placeholder with a genuine 1-2 sentence answer.

src/run_pipeline.py refuses to run at all while any placeholder remains ? that's the same "must actually be filled in" rule your generate_model_card function enforces, applied to you first.

```
`bash
python src/run_pipeline.py
`
```

It registers both candidates, deliberately attempts two promotions that should be blocked (no card, then f1 too low) and prints why each was blocked, then successfully promotes the model that clears the bar and writes registry_summary.json.

Algorithm / Logic Explanation

1. register_model ? Artifact Store with Immutable Versioning

Purpose: Register a new model version as an immutable artifact in the registry.

Algorithm:

1. Version Allocation: Uses _next_version_id() to scan existing version directories (v1, v2, ...) under registry_dir/models/{name}/ and returns the next sequential version ID (e.g., v1, v2, v3). This ensures:

- No overwriting of prior versions (immutability)
- Deterministic, monotonic version numbering
- O(n) scan where n = number of existing versions

2. Directory Creation: Creates the version-specific directory at registry_dir/models/{name}/{version_id}/ using _model_dir() helper.

3. Artifact Copy: Reads the model file from model_path as JSON and writes it to {version_dir}/model.json ? this is the immutable model artifact.

4. Manifest Creation: Writes a manifest.json in the version directory containing:

- version_id: The allocated version string
- name: Model name
- metrics: The metrics dict passed in (precision, recall, f1, auc)
- stage: Initial value "None" (not in any deployment stage)
- created_at: ISO-8601 UTC timestamp from _now()

5. Returns: The version_id string for caller reference.

Key Design Decision: The manifest is separate from the model artifact, allowing metadata (stage, history) to be updated without modifying the immutable model file.

2. generate_model_card ? Governance Record with Validation

Purpose: Create a model card (governance document) for a registered version, enforcing completeness.

Algorithm:

1. Validation Loop: Iterates through REQUIRED_CARD_FIELDS = ["intended_use", "training_data", "limitations", "ethical_considerations"]:
 - Checks field exists in card_fields dict
 - Checks value is non-empty/non-whitespace
 - Checks value does NOT contain substring "TODO" (catches placeholder text)
 - Raises ValueError with specific field name on any failure
2. Metrics Retrieval: Reads the version's manifest.json to pull the metrics object ? ensures the card reflects the actual registered metrics.
3. Card Assembly: Constructs the card JSON with:
 - name, version_id: Identity
 - All 4 fields from card_fields
 - metrics: From manifest
 - created_at: Current timestamp
4. Persistence: Writes to {version_dir}/model_card.json with pretty-printed JSON.
5. Returns: Path to the written card file.

Key Design Decision: Validation happens before any I/O ? fail fast if card is incomplete. The "TODO" check prevents students from submitting placeholder content.

3. promote_model ? Governance Gate with Audit Trail

Purpose: Move a model version between stages ("Staging" or "Production"), enforcing production gates.

Algorithm:

1. Load Manifest: Reads the version's manifest.json.
2. Production Gate Checks (only if target_stage == "Production"):
 - Card Existence: Verifies model_card.json exists in version directory using os.path.exists(). Raises GovernanceError if missing.
 - F1 Threshold: Compares manifest["metrics"]["f1"] against PRODUCTION_F1_THRESHOLD (0.70). Raises GovernanceError with actual f1 value if below threshold.
 - Staging promotions skip both checks.
3. Auto-Archive Prior Production (only for Production promotion):
 - Scans all other versions of the same model in registry_dir/models/{name}/
 - For each other version, reads its manifest
 - If any has stage == "Production", updates its stage to "Archived" and writes back its manifest
 - Ensures at most one Production version exists at any time
4. Stage Transition & History:
 - Records old_stage = manifest.get("stage", "None")
 - Updates manifest["stage"] = target_stage
 - Appends to manifest["history"] (creates list if missing): {"from_stage": old_stage, "to_stage": target_stage, "at": _now()}
 - Never overwrites history ? preserves full audit trail
5. Persist & Return: Writes updated manifest back to disk, returns the updated manifest dict.

Key Design Decisions:

- Gates are enforced in code, not documentation ? impossible to bypass
- Auto-archive prevents "which version is in production?" ambiguity
- History list provides immutable audit trail for compliance

4. get_production_model ? Source of Truth Query

Purpose: Return the currently deployed Production model manifest, or None.

Algorithm:

1. Lists all version directories under registry_dir/models/{name}/
2. For each version, reads its manifest.json
3. Returns the first manifest where manifest.get("stage") == "Production"
4. Returns None if no version has stage "Production"

Complexity: $O(n)$ where n = number of versions. Trivial for small n (2-40), would need an index for 1000+ versions.

Key Design Decision: Scans manifests on-demand rather than maintaining a separate index ? simple, consistent, no cache invalidation issues.

Pipeline Flow (run_pipeline.py)

The driver script demonstrates the complete governance flow:

- 1. Register candidate_a (f1=0.495) ? v1
- 2. Register candidate_b (f1=0.899) ? v2
- 3. Try promote v1 to Production (NO card) ? BLOCKED: "no model_card.json"
- 4. Generate card for v1
- 5. Try promote v1 to Production (card exists, f1=0.495 < 0.70) ? BLOCKED: "f1 below threshold"
- 6. Generate card for v2
- 7. Promote v2 to Production ? SUCCESS (f1=0.899 >= 0.70)
- 8. Query production ? returns v2 manifest
- 9. Write registry_summary.json

Self-check

```
`bash
pytest tests/ -q`
```

This is a self-check, not the grader.

Deliverables (what you commit)

- src/mini_model_registry.py, completed.
- model_card_fields.json, filled in with real content (no TODO left).
- The .model_registry/ directory your pipeline run produced (small JSON manifests + cards only).
- registry_summary.json.
- A short NOTES.md: the --student-id value you used (required ? see above), which candidate ended up in Production and why, what would you need to add to promote_model's gate if you also wanted to block promotion of a model trained on stale (e.g. >30-day-old) feature data, and ? tying back to this week's AutoML/HPO framing ? if a hyperparameter search had handed you 40 candidates instead of 2, what in your register_model/promote_model design would need to change (or genuinely wouldn't) to gate 40 instead of 2?

Grading checklist

- [] data/ matches what generate_for_student.py --student-id <NOTES.md value> actually produces.
- [] register_model correctly versions models and never overwrites a prior version.

- [] generate_model_card genuinely rejects incomplete/TODO cards (checked against a held-out incomplete card, not just the one you tested with).
- [] promote_model blocks Production promotion on both governance conditions independently, and correctly archives the prior Production version on a successful promotion.
- [] get_production_model returns the right version, and None when nothing is in Production.
- [] model_card_fields.json is genuinely filled in ? no leftover TODO, answers are specific to this model, not generic filler.
- [] NOTES.md shows real reasoning about the second and third questions, not just a restatement of the first.
- [] Meaningful commit history and a working README.

Submission

```
`bash
git add -A
git commit -m "Week 5: model registry governance"
git tag week05-submit
git push origin main --tags
`
```

Outputs from This Run (Student ID: 142301024)

```
`
$ python generate_for_student.py --student-id 142301024
student_id: 142301024
seed: 3585038552
candidate_a: f1=0.495 (below 0.70 bar)
candidate_b: f1=0.899 (clears 0.70 bar)

$ python src/run_pipeline.py
Registered candidate_a as v1, candidate_b as v2
[expected] promoting v1 with no card was blocked: cannot promote fraud-detector v1 to Production: no model_card.json
[expected] promoting v1 with f1 below threshold was blocked: cannot promote fraud-detector v1 to Production: f1=0.495 < 0.7
Promoted v2 to Production. History: [{'from_stage': 'None', 'to_stage': 'Production', 'at': '2026-09-01T15:42:16.090611+00:00'}]
Wrote registry_summary.json: production version is v2

$ pytest tests/ -q
..... [100%]
8 passed in 0.06s
`

registry_summary.json:
`json
{
  "model_name": "fraud-detector",
  "production_version": "v2",
  "production_metrics": {
    "precision": 0.916,
    "recall": 0.883,
    "f1": 0.899,
    "auc": 0.968
  }
}
`
```

Production Model: candidate_b (version v2) with f1=0.899 ? the only candidate that clears the 0.70 production quality bar.